

Разрабатываю .NET backend-системы, где платежи, доступ и состояние должны переживать retries и restarts: explicit state machines, provider re-verification, durable inbox/outbox, idempotency, reconciliation, migrations и проверяемый rollback

Payment correctness

Webhook не считается источником истины: provider payment повторно запрашивается и проверяется по id/status/environment/metadata/amount/currency

Recovery ownership

Durable inbox/outbox, idempotency, reconciliation, restart recovery, leases и manual review для конфликтующих evidence

Release safety

Versioned releases, coordinated backup, atomic switch, health gates и rollback на предыдущую версию

ОПЫТ

- 2026 — сейчас

CompilationLabLMS — payment/backend architecture

 - Сделал PaymentOrder владельцем допустимых state transitions, retry scheduling и status-check leases; webhook flow повторно запрашивает payment у provider перед внутренним событием
 - Проверяю payment id/status/environment/metadata/amount/currency и отделяю malformed/unverifiable provider events от подтверждённого состояния
- 2026 — сейчас

VpnMediator — state/recovery backend

 - Спроектировал durable payment inbox/outbox, idempotent reconciliation и manual-review path для конфликтующих provider evidence; recovery после restart проверяется отдельными тестами
 - Построил versioned release path с coordinated backup, atomic switch, readiness/liveness gates и возвратом на предыдущий release при неуспешном запуске
- 2024 — сейчас

UniversalToolchain — .NET platform architecture

 - Применяю тот же подход к platform code: single-owner planning, exact package binding и regression contracts; текущий exact manifest — 1 306/1 306 passed

ПРОЕКТЫ

- CompilationLabLMS

Архитектура отделяет пользовательские workflows от финансовых инвариантов и поддерживает безопасную миграцию и откат.
- VpnMediator

Production-сервис с контролируемыми переходами состояния, backup/restore, health gates и безопасным rollback; публичный разбор не раскрывает секреты и пользовательские данные.
- UniversalToolchain

LanguageRuntime проверяет exact planned package/component graph и создаёт выбранные компоненты без повторного планирования; контракт защищён determinism/exact-binding checks, ownership guards и interpreter/CIL parity.

КОМПЕТЕНЦИИ

- NET Backend

C#/.NET, ASP.NET Core, REST/OpenAPI, webhooks, background services, provider integrations
- State & Data

PostgreSQL, SQLite, transactions, explicit state machines, idempotency keys, inbox/outbox, leases
- Reliability

provider re-verification, reconciliation, retry ownership, restart recovery, audit/manual review, backup/restore
- Operations / Platform

Linux, Docker Compose, nginx, systemd, health gates, versioned releases, atomic switch, rollback

ОБРАЗОВАНИЕ

- НИУ ВШЭ — Программная инженерия, 2026–2030

Диплом I степени и Главная премия «Совершенство как надежда» за UniversalToolchain

Москва / удалённо